

Hotel Transilvania

Gestion de usuarios y roles (ms-auth-usuarios-service)

Documento de referencia para el equipo. Explica como se crean usuarios, que rol se asigna en cada caso y como obtener un usuario ADMIN sin editar la base de datos (cuando ya existe un administrador).

Fecha de referencia: mayo 2026 | Puerto del servicio: 8081

1. Resumen

- El registro publico (/auth/register) siempre crea usuarios con rol USER.
- Un ADMIN se crea manualmente.

2. Microservicio y ubicacion en el codigo

Tanto /auth/register (siempre USER) como /usuarios (crear ADMIN con token ADMIN) pertenecen al MISMO microservicio. No hay un MS separado para /usuarios.

Microservicio: ms-auth-usuarios-service

Puerto: 8081

Base de datos: auth_usuarios_db (tabla usuario)

Modulo Maven: ms-auth-usuarios-service/

2.1 Mapa endpoint -> controlador -> servicio

HTTP / Ruta	Controlador	Metodo Java	Servicio	Rol / acceso
POST /api/v1/auth/register	AuthController	register()	UsuarioService.register()	Siempre USER. Publico.
POST /api/v1/auth/login	AuthController	login()	AuthenticationManager + JwtService	Publico. Devuelve JWT.
GET /api/v1/auth/validate	AuthController	validateToken()	-	Requiere token valido.
POST /api/v1/usuarios	UsuarioController	save()	UsuarioService.save()	USER o ADMIN. Solo token ADMIN.
PUT /api/v1/usuarios/{id}	UsuarioController	update()	UsuarioService.update()	Cambiar rol. Solo ADMIN.
GET /api/v1/usuarios	UsuarioController	findAll()	UsuarioService.findAll()	Solo ADMIN.
GET /api/v1/usuarios/{id}	UsuarioController	findById()	UsuarioService.findById()	USER o ADMIN.

DELETE /api/v1/usuarios/{id}	UsuarioController	delete()	UsuarioService.delete()	Solo ADMIN.
------------------------------	-------------------	----------	-------------------------	-------------

2.2 Rutas de archivos en el proyecto

ms-auth-usuarios-service/src/main/java/.../controller/

AuthControlle

Resumen: /auth/* es autenticacion y registro publico. /usuarios/* es CRUD de usuarios con @PreAuthorize. Ambos comparten UsuarioService y la misma BD.

3. Registro publico (solo USER)

Endpoint: POST /api/v1/auth/register

No requiere token. El cuerpo solo admite nombre y contraseña (RegisterRequest).

En UsuarioService.register() el rol se fuerza a USER antes de guardar. Aunque se intente enviar otro rol por otro medio, este endpoint no lo acepta.

POST http://localhost:8081/api/v1/auth/register

Content-Type:

4. Creacion por administrador (USER o ADMIN)

Endpoint: POST /api/v1/usuarios

Requiere: Authorization: Bearer <token de usuario ADMIN>

El cuerpo incluye el campo rol (USER o ADMIN).

POST http://localhost:8081/api/v1/auth/login

{ "nombre": "a

5. Cambiar rol de un usuario existente

Endpoint: PUT /api/v1/usuarios/{id}

Solo ADMIN. Permite actualizar nombre, contraseña (opcional) y rol.

PUT http://localhost:8081/api/v1/usuarios/2

Authorization:

Nota: si password va vacío, conviene omitir el campo o no cambiarla según la lógica del servicio (solo se codifica si no está en blanco).

6. Tabla comparativa

Acción	Solo USER?	Sin tocar BD?	Quien puede	Microservicio
POST /auth/register	Si	N/A	Cualquiera	ms-auth :8081
POST /usuarios	No	Si	Solo ADMIN	ms-auth :8081
PUT /usuarios/{id}	No	Si	Solo ADMIN	ms-auth :8081
Usuario USER	No puede ADMIN	No	USER	-
Script SQL / phpMyAdmin	No	Manual	MySQL local	auth_usuarios_db

7. Por que a veces se usa la base de datos

Si el equipo solo usa /auth/register o aun no existe ningun usuario ADMIN (primer arranque), no hay forma de llamar a POST /usuarios con permisos de administrador. En ese caso es normal:

- Ejecutar el script scripts/sql/run-seed.ps1 (usuario admin / admin123).

- O insertar

8. Usuarios de prueba (seed)

- admin / admin123 -> rol ADMIN

- user / us

9. Seguridad (motivo del diseno)

Antes era posible registrarse como ADMIN enviando rol en el body del register. Eso se corrigio: el registro publico solo crea USER para evitar escalada de privilegios. La gestion de ADMIN queda en endpoints protegidos.